Table Logic: A Relational Database and Real-Time Seating Tracker

Aaron Graf, Dr. Valerie Ann Haywood, and Aaron Tobias

MSC 5308: Software Engineering

College of Arts and Sciences: Concordia University Texas

for

Dr. Moody Amakobe

Version 1.0

May 10, 2026

Problem Statement

The main challenges in the hospitality industry often stem from relying on manual, paper-based processes, which can create communication gaps and increase the likelihood of human error (Lupasc & Huzum, 2020; Naidu et al., 2024). When data is not stored in a central system, it can lead to disorganized records and mistaken wait-time estimates. These issues contribute to market saturation and increase the likelihood that customers will switch, which can be quite costly (Adroit Market Research, 2023; Pymnts, 2024).

From a software engineering perspective, these manual processes signal a challenge in ensuring system dependability and optimal resource management. When there is no automated synchronization between front-of-house seating and backend resource availability, organizations may incur higher failure costs, including inefficient table turnover and lost revenue during peak periods (Sommerville, 2016). TableLogic tackles these issues with a robust relational schema and real-time tracking interface, bridging front-of-house and backend data gaps.

Target Users

User Type	Description	Primary Goal	Technical Level	Usage Frequency
Host/Manager	Staff managing active seating and guest flow.	Optimize table turnover and reduce guest waiting times.	Moderate	Daily
Restaurant Owner	Stakeholders focused on business intelligence.	Analyze operational performance and labor metrics	Low-Moderate	Weekly
Restaurant Attendee	End customers seeking dining reservations.	Secure a reservation in time and receive accurate wait-time estimates.	High	Monthly

Table 1. *TableLogic Target User Profiles: Goals, Technical Proficiency, and Interaction Frequency.*

High-Level Features

- *Feature 1: Interactive Table Map:* A digital floor plan displaying real-time table availability (Available vs. Occupied).
- *Feature 2: Host Control Dashboard:* A secure interface allowing restaurant staff to manually toggle table statuses as guests arrive and leave.
- *Feature 3: Capacity Filtering:* A feature allowing the host to filter available tables by party size (e.g., showing only 4-tops).
- *Feature 4: Data Persistence:* A relational database system that ensures real-time synchronization between the host stand and the backend server to prevent double-seating.

Technology Stack

Layer	Technology	Version	Rationale
Frontend	React.js	18.x	Team familiarity: the component model is a perfect fit for our interactive table map UI.
Backend	Node.js + Express	20.x LTS	Using JavaScript throughout the stack reduces context switching and shortens the learning curve for the team.
Database	PostgreSQL	15.x	A robust relational model is a perfect fit for our rigid data entities (Tables, Capacity, Status).
Hosting	Render	-	The free tier covers our needs. It automatically deploys from GitHub.
Continuous Integration/ Continuous Delivery	GitHub	-	Offers native GitHub integration without needing additional accounts.
Testing	Jest + Supertest	-	The industry standard for Node applications; easy to integrate into our CI pipeline.
Alternative Database:	Microsoft SQL Server	-	Using MS SQL alongside Python and Node creates "Tech Stack Soup". Consolidating to PostgreSQL keeps our architecture strictly aligned with the standard PERN stack, significantly reducing the learning curve.
Alternative Database:	Microsoft Power BI	-	Power BI is a read-only data analysis tool. It cannot easily provide the clickable buttons our Host Control Dashboard requires to change table statuses. React satisfies the rubric's requirement for a true Frontend layer.

Table 2. *System Architecture: Integrated Technology Stack and Developmental Rationale.*

Success Criteria

#	Success Criterion	How it Will Be Verified
1	**Core Implementation:** The Interactive Table Map and Host Control Dashboard are fully functional.	Demonstrated in the Week 16 live demo.
2	**Public Accessibility:** The application is deployed to a public URL and accessible 24/7.	Live URL submitted in Week 10 with uptime monitoring enabled.
3	**Data Synchronization:** The frontend accurately and consistently reflects backend database changes (Available vs. Occupied).	Verified via Postman test suite and OpenAPI spec.
4	**CI/CD Pipeline**	Visible in the GitHub Actions run history.
5	**Technical Reproducibility**	Verified by a peer or instructor attempting to set up cold.
6	**Audit Traceability:** 100% of code changes are documented via GitHub using conventional commits.	Visible in the GitHub commit history and PR reviews.

Table 3. *Measurable Performance Benchmarks and Quality Assurance Metrics*

Assumptions and Constraints

Category	Description
Assumption	Users (both restaurant staff and customers) will have access to a modern web browser and a stable internet connection.
Assumption	Team members will develop sufficient familiarity with the chosen PERN tech stack (PostgreSQL, Express, React, Node.js) over the course of the semester.
Constraint	The project must be completed and fully documented within the 16-week course timeframe.
Constraint	The system must be built using only free-tier or open-source tools (e.g., Render's free hosting tier, open-source PostgreSQL).
Constraint	All code integrations must follow strict version control, requiring a pull request and at least one approving review before merging to the `main` branch.

Table 4. *Project Guardrails: Structural Constraints and Environmental Assumptions.*

Out of Scope

To ensure the project's feasibility and high reliability within the set timeframe, the following items are explicitly excluded from the scope.

1. *Future Date/Time Reservations:* To ensure MVP feasibility, the app will strictly handle real-time availability for walk-in customers, avoiding complex calendar and timezone logic.
2. *Financial Transaction Processing:* The system will track table status but will not handle payments or integrate with third-party payment gateways (e.g., Stripe, Square).
3. *Native Mobile Application:* The project will be built as a responsive web application; no native iOS or Android mobile applications will be developed.
4. *Real-Time IoT Integration:* The application will rely on manual host inputs and will not interface with physical hardware, such as automated Kitchen Display Systems or smart tables.

Team Roles and Working Norms

Name	Primary Role	Secondary Responsibilities
Aaron T.	Project Lead/ DevOps	Facilitates agile standups, manages the GitHub repository, configures the Render hosting environment, and ensures the integrity of the CI/CD pipeline.
Aaron G.	Frontend Lead	Develops the interactive React.js table map UI and the host control dashboard, ensuring a clean and responsive user experience.
Valerie H.	Backend Lead	Designs the PostgreSQL database schema and develops the Node.js/Express API routes to handle real-time table status updates.

Table 5. *Project Governance: Defined Roles, Responsibilities, and Standards.*

Working Norms

- *Meetings:* Saturdays at 11 AM via Microsoft Teams. Flexible asynchronous standups would occur three times weekly on a rolling basis via Microsoft Teams to accommodate varying schedules.
- *Communication channel:* Microsoft Teams
- *Branch naming convention:* feature/short-description, fix/issue-description
- *Commit message format:* Conventional Commits — feat: , fix: , docs: , chore:
- *PR review turnaround:* All PRs must receive at least 1 review within 24 hours
- *Decision-making:* The team uses a consensus model for key architectural and design decisions, ensuring that every member's technical perspective is integrated before implementation.

Document Version Ledger

Version	Date	Author	Changes
1.0	5/10/2026	Team	Initial Draft
1.1		Team	Updated after requirements and design work (to be completed in Week 5).

Table 6. *Version Control Matrix: Document Iterations, Authorship, and Chronological Change Log.*

Ethical Use of AI

As a novice in the field of computer science with no prior professional industry experience, we used Gemini (AI) and Grammarly to support the development of this Vision Document and its underlying technical architecture. Specifically:

- *Technical Guidance & Troubleshooting:* We relied on Gemini to navigate environment setup and troubleshoot configurations for Node.js, React, and PostgreSQL. This helped

bridge the gap between my academic learning and the practical application of these tools.

- *Structural & Technical Writing:* Gemini was used to help outline the technical report and draft complex sections, such as the Success Criteria and Technology Stack, ensuring that the document adhered to professional software engineering standards (Sommerville, 2016).
- *Documentation Standards:* We used Gemini to ensure that all headings, table captions, and citations adhere strictly to APA 7th Edition formatting requirements.
- *Editing & Clarity:* Grammarly was employed to review the document for grammatical precision, tone consistency, and overall readability, ensuring the final deliverable meets graduate-level communication standards.

All final decisions regarding the system's architecture, selected features, and project governance were made by the authors to ensure the TableLogic system aligns with the specific goals of the MCS 5308 course.

References

Adroit Market Research. (2023). *Restaurant online ordering system market trends and growth drivers: Strategic analysis of Restolabs, Olo, MenuDrive, Toast POS, and ChowNow.* https://www.adroitmarketresearch.com/industry-reports/restaurant-online-orderingsystem-market

Conger, S. (2014). *Hands-On Database* (2nd ed.). Pearson.

Hernandez, M. J. (2013). *Database design for mere mortals: A hands-on guide to relational database design* (3rd ed.). Addison-Wesley.

Lupasc, A., & Huzum, M.-C. (2020). Designing an information system for managing restaurant orders. Annals of Dunarea de Jos University. *Fascicle I: Economics and Applied Informatics, 26(2)*, 194–199.

Microsoft. (n.d.). What is SQL Server? Microsoft Learn. Retrieved from https://learn.microsoft.com/enus/sql/sqlserver/what-is-sql-server?view=sql-server-ver17

Naidu, M. S. S., Pravin, A., Rao, I. R. S. N., & Sivaranjani, K. (2024). Embedded system-based restaurant automation. *International Journal of Emerging Research in Engineering, Science, and Management, 3(4)*, 29–32. https://doiorg.concordia.idm.oclc.org/10.58482/ijeresm.v3i4.4

Pymnts. (2024). *Starbucks Uses AI-Powered Personalized Rewards to Boost Frequency and Spend.* Retrieved from https://www.pymnts.com/news/loyalty-and-rewardsnews/2024/starbucks-uses-artificial-intelligence-powered-personalized-rewards-boostfrequency-check-size/

Tech Monitor Staff Writer. (2023, January 6). What is SQL Server? Tech Monitor. Retrieved from https://www.techmonitor.ai/what-is/what-is-sql-server

TechTarget. (n.d.). Microsoft SQL Server. SearchDataManagement. Retrieved from https://www.techtarget.com/searchdatamanagement/definition/SQL-Server

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.